REPORT

Reviewer report: Textualize/rich#4177: Optimize syntax

word wrap without line numbers

Run rich-4177, tested 2026-11-08 12:41 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 12:03 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Optimize syntax word wrap without line numbers (Textualize/rich#4177).

• What we observed: could not verify: claim(s) C2 expect a difference but no observation matches (25 test(s) ran

on main, 26 on the PR; 1 new test(s) pass, 19 unchanged; 6 test failure(s) pre-existing and identical on both

sides; probe model-probe1 is not_adequate (the probe crashed or printed no parsed output on every main-side

run; not counted as ran or adequate); its matching results do not count).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 6.6 GB (of 15.6 GB).

• Setup check: free disk 93.0 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: rich python setup install instructions https://github.com/Textualize/rich site:github.com/Textualize/rich

• Query: rich python documentation install https://github.com/Textualize/rich site:github.com/Textualize/rich

• Query: rich python install requirements https://pypi.org/project/rich/ https://github.com/Textualize/rich

site:pypi.org

• Source: rich - PyPI — https://pypi.org/project/rich/

• Source: rich-cli - PyPI — https://pypi.org/project/rich-cli/

• Source: rich-pyfiglet - PyPI — https://pypi.org/project/rich-pyfiglet/

• Source: GitHub - Textualize/rich: Rich is a Python library for rich text and beautiful formatting in the terminal. ·

GitHub — https://github.com/textualize/rich

• Source: [BUG] rich fails to install using pip in editable mode · Issue #346 · Textualize/rich · GitHub —

https://github.com/Textualize/rich/issues/346

• Source: GitHub - Textualize/rich-cli: Rich-cli is a command line toolbox for fancy output in the terminal · GitHub

— https://github.com/textualize/rich-cli

• Source: rich/README.ru.md at main · Textualize/rich · GitHub —

https://github.com/textualize/rich/blob/master/README.ru.md

• Source: rich/.readthedocs.yml at main · Textualize/rich · GitHub —

https://github.com/Textualize/rich/blob/main/.readthedocs.yml

• Setup plan: the same packages on main and pr.

• Setup plan: install alabaster==1.0.0 (listed in requirements.txt) on main and pr.

• Setup plan: install Sphinx==8.2.3 (listed in requirements.txt) on main and pr.

• Setup plan: install sphinx-rtd-theme==3.0.2 (listed in requirements.txt) on main and pr.

• Setup plan: install sphinx-copybutton==0.5.2 (listed in requirements.txt) on main and pr.

• Setup plan: install poetry (listed in tox.ini deps) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9.0

• Setup plan: refused rich-cli: its PyPI page does not link to this GitHub repository

• Installed alabaster 1.0.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed Sphinx 8.2.3 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed sphinx-rtd-theme 3.0.2 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed sphinx-copybutton 0.5.2 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed poetry 2.5.1 from tox.ini (listed in tox.ini deps) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (25 unchanged, 3 runs per side) | pytest-pr-1.log:243 «6 failed, 20 passed in

1.28s»

| P-C1 | The rendered output of Syntax(word_wrap=True, line_numbers=False) is unchanged by the

optimization. (PR description) | not testable | probe model-probe1 has no observation for this claim | no matching

log line

| P-C2 | The optimization reduces the number of calls to console.render_lines(...) for Syntax(word_wrap=True,

line_numbers=False). (PR description) | not testable | probe model-probe1 has no observation for this claim | no

matching log line

Citation check (WARN): 1 citation(s) verified; 2 statement(s) have no matching log line.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 25 / 26 | | Tests unchanged | 25 | -

| New tests that pass | 1 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 6 | test_syntax::test_option_no_wrap,

test_syntax::test_python_render, test_syntax::test_python_render_indent_guides,

test_syntax::test_python_render_simple, test_syntax::test_python_render_simple_passing_lexer_instance

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe1.py, frozen

b56d015ef468 at 2026-11-08T07:13:00.342290+00:00): Check output consistency and render_lines call count

for Syntax with word_wrap=True and line_numbers=False

• P-C1 (PR description): The rendered output of Syntax(word_wrap=True, line_numbers=False) is unchanged by

the optimization. Expects the covered observations to stay the same.

• P-C2 (PR description): The optimization reduces the number of calls to console.render_lines(...) for

Syntax(word_wrap=True, line_numbers=False). Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| probe.error | - | probe could not run | same on both sides: {"crash": "non-json output", "exit_code": 1} |

probe-model-probe1-main-1.jsonl:1 «"observation": "probe.error"»
probe-model-probe1-pr-1.jsonl:1

«"observation": "probe.error"»

Citation check (PASS): 2 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (4.5 s run).

Coverage gate (python, confidence normal): 6 of 6 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| rich/syntax.py | 6 | 6

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested c651c4fb6a91 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: pytest exit 1; 6 failure(s)/error(s) pre-existing and identical on both sides.

• Changed code was exercised (coverage gate): 6 of 6 changed executable line(s) ran (100%).

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every probe observation cites its output line: 2 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | could not verify: claim(s) C2 expect a difference but no observation

matches (25 test(s) ran on main, 26 on the PR; 1 new test(s) pass, 19 unchanged; 6 test failure(s) pre-existing

and identical on both sides; probe model-probe1 is not_adequate (the probe crashed or printed no parsed output

on every main-side run; not counted as ran or adequate); its matching results do not count) | Read the

setup/build log under Evidence links; build failures usually need a compiler or a newer nightly.

| Probes ran on both sides | WARN | model-probe1 main#1; model-probe1 pr#1; model-probe1 main#2 | Read

the probe-*.log files; a probe that cannot run is not evidence either way.

| Every claim cites a real log line | WARN | 1 citation(s) verified; 2 statement(s) have no matching log line |

Regenerate the report; if it persists, the logs were edited or truncated.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | pytest exit 1; 6 failure(s)/error(s) pre-existing and identical on both sides

| Changed code was exercised (coverage gate) | PASS | 6 of 6 changed executable line(s) ran (100%)

| Probes ran on both sides | WARN | model-probe1 main#1; model-probe1 pr#1; model-probe1 main#2

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | WARN | 1 citation(s) verified; 2 statement(s) have no matching log line

| Every probe observation cites its output line | PASS | 2 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-08/rich-4177 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/Textualize/rich/pull/4177

Evidence links

• Pull request Textualize/rich#4177

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-probe1-main-1.jsonl

• probe-model-probe1-pr-1.jsonl

• main run 1 result: pytest-main-1.log:243 «6 failed, 19 passed in 1.30s»

• pr run 1 result: pytest-pr-1.log:243 «6 failed, 20 passed in 1.28s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 12:03 IST